

1. After reviewing the Google Big Table papers, provide an answer to the following questions.

Q1.1 The Google SSTable file format is used to store Bigtable data. What does an SSTable internally contain? What are the core steps of an SSTable lookup?

Internally, an SSTable contains a sequence of blocks (typically 64KB in size) and a block index stored at the end of the file, which is loaded into memory when the SSTable is opened. The core steps for a lookup involve a single disk seek: the system first performs a binary search in the in-memory block index to find the correct block, and then reads that specific block from the disk.

Q1.2 BigTable heavily relies on the Chubby lock service. Explain what Chubby is, and why and how it is used by BigTable.

Chubby is a highly-available, persistent distributed lock service consisting of five active replicas (one elected as master) that provides a namespace of directories and small files that can be used as atomic locks. Bigtable uses Chubby to ensure there is at most one active master at any time, to store the bootstrap location of Bigtable data, to discover and track live tablet servers, to store table schema information, and to store access control lists.

Q1.3 How is a BigTable split among multiple files?

A Bigtable is dynamically partitioned into row ranges called tablets. Initially, a table consists of just one tablet, but as the table grows, it is automatically split into multiple tablets of approximately 100-200 MB in size by default. These tablets act as the basic unit of distribution and load balancing across different tablet servers.

Q1.4 What are the responsibilities of the BigTable Master server and of the Tablet servers?

The Master Server is responsible for assigning tablets to tablet servers, detecting when tablet servers join or expire, balancing tablet-server load, garbage collecting files in GFS, and handling schema changes (like creating tables and column families). Instead Tablets servers are responsible for managing a set of tablets, handling incoming read and write requests for the tablets they load, and splitting tablets when they grow too large

Q1.5 How are the Tablet locations stored and handled?

Tablet locations are stored using a three-level hierarchy similar to a B+-tree.

1. A file in Chubby stores the location of the root tablet.
2. The root tablet (which is never split) stores the locations of all other METADATA tablets.
3. Each METADATA tablet stores the locations of the actual user tablets. Clients cache tablet locations locally. If a client does not know a location or its cache is stale, it recursively moves up this hierarchy to discover the correct tablet location, eventually prefetching and caching the newly discovered metadata.

Q1.6 How does the master node discover Tablet servers? And how does it check if a Tablet server is no longer serving the assigned Tablets?

The master node discovers tablet servers by monitoring a specific directory (the servers directory) in Chubby. To check if a tablet server is no longer serving its assigned tablets, the master periodically asks each tablet server for the status of its lock.

Q1.7 What are the actions performed by the Big Table Master if it discovers a Tablet server fails?

If a tablet server reports losing its lock or cannot be reached, the master attempts to acquire an exclusive lock on that server's file in Chubby. If successful, the master deletes the server's file to ensure it can never serve again, and then moves all the tablets previously assigned to that server into the set of unassigned tablets for reassignment.

Q1.8 What does the Master do if the connection with Chubby is disrupted? Does this failure impact the assignment of Tablets to Tablet servers?

If its Chubby session expires, the master kills itself to prevent vulnerabilities stemming from network issues. However, this master failure does not change or impact the assignment of tablets to tablet servers.

Q1.9 What are the steps executed by the BigTable Master at startup?

At startup, the master executes four steps:

1. Grabs a unique master lock in Chubby to prevent concurrent masters.
2. Scans the servers directory in Chubby to find live servers.
3. Communicates with every live tablet server to discover what tablets are already assigned.
4. Ensures the *root tablet* is assigned, and then scans the METADATA table to learn the set of all existing tablets, adding any unassigned tablets to the pool of eligible tablets for assignment.

Q1.10 When does the set of existing tablets change?

The set of existing tablets only changes when a table is created or deleted, when two existing tablets are merged to form one larger tablet, or when an existing tablet is split into two smaller tablets

Q1.11 How do the read and write operations at a Tablet server work? How a Tablet server recovers a Tablet?

Incoming write operations are checked for proper formatting and authorization. Valid mutations are appended to a commit log (using group commit for throughput) and their contents are then inserted into an in-memory sorted buffer called a memtable.

Read operations are similarly checked for validity and authorization. They are executed on a merged view of the sequence of on-disk SSTables and the memtable.

To recover a tablet, the server reads the tablet's metadata from the METADATA table, which contains a list of SSTables and "redo points" (pointers into the commit logs). The server reconstructs the state by reading the SSTable indices into memory and rebuilding the memtable by applying all committed updates from the log since the redo points.

Q1.12 BigTable maintains a single commit log for each tablet server. How is it implemented and handled to enable an efficient recovery when the tablets of a failed tablet server are reassigned to many tablet servers?

To optimize write performance, mutations for all tablets managed by a server are co-mingled and appended to a single physical commit log file. To prevent newly assigned tablet servers from reading the entire log during recovery, the master coordinates a process that sorts the commit log entries by the keys <table, row name, log sequence number>. This sorts the data so that all mutations for a specific tablet are contiguous, allowing the new server to recover its data efficiently using a single disk seek followed by a sequential read.

2. After reviewing the Dynamo papers, provide an answer to the following questions

Q2.1 Why is partitioning introduced, and how is it implemented?

Partitioning is introduced to allow the system to scale incrementally as demand grows. It is implemented using consistent hashing, where the output range of a hash function is treated as a fixed circular space, or "ring".

Q2.2 How are data assigned to storage nodes? How is the subset of data a node is responsible for determined?

Data is assigned by hashing the data item's key to yield its position on the ring. The system then walks the ring clockwise from that position to find the first node with a larger position value; that node becomes responsible for the data. Thus, a node is responsible for the region of the ring between itself and its immediate predecessor node.

Q2.3 What are the concerns in using consistent hashing, and how are they resolved in Dynamo?

The main concerns with basic consistent hashing are that random node placement leads to non-uniform data and load distribution, and it is unaware of the heterogeneity of different nodes' hardware performance. Dynamo resolves this by introducing "virtual nodes". Instead of mapping

to a single point, each physical node is assigned multiple positions (tokens) on the ring based on its capacity, ensuring a more balanced and proportional load distribution.

Q2.4 Which data replication strategy is used in Dynamo? How can it be guaranteed that the N replicas of data are stored on distinct physical nodes?

Dynamo replicates each data item on N hosts. The coordinator node stores the key locally and also replicates it to the N-1 clockwise successor nodes in the ring. Because of virtual nodes, the first N successor positions might be owned by fewer than N physical machines; to guarantee distinct physical nodes, Dynamo constructs a preference list by skipping positions on the ring until it finds N uniquely distinct physical hosts.

Q2.5 Why is data versioning introduced, and how is it implemented? When are two changes considered to be in conflict and require reconciliation?

A get() operation may not return the last written data. At the same time, we want to make sure to now overwrite and lose an earlier modification.

Dynamo uses versioning, and each modification is a new immutable version of the data, and each version is identified using a vector clock.

Most of the time, new versions subsume previous versions and the system itself is able to determine the authoritative version.

But in some cases (two concurrent writes on the data read before the first of the two writes) it may not be possible. In these cases, the get() operations returns all the unreconciled versions, and the client itself has to do the reconciliation.

Q2.6 What are the possible actions performed by a node that receives a put request? When the write operation is considered successful?

Upon receiving a put(), the coordinator generates the vector clock and writes it locally. Then it forwards the new version along with the new vector clock to the N highest-ranked reachable nodes (in the preference list for that key). The operation is successful if at least W-1 (W is a configurable parameter) nodes acknowledge the write.

If a client request is routed through a generic load balancer, it may arrive at a node that is not among the top N nodes in the key's preference list. In this case, the node will forward it to the first among the top N in the key's preference list.

If one of top N nodes are unavailable due to server outages or network partitions, the system engages "hinted handoff". The request is sent to the first N healthy nodes encountered while walking the ring, even if they are not the intended recipients (some of them are after the first N), they will be stored in a separate local db and they are sent together with a hint that tells the correct node it should be placed.

When the offline node comes back, hinted replicas are properly moved there (copied and then deleted to never decrease replica count).

Q2.7 What is a hinted replica? How does hinted handoff work?

As I answered above, a hinted replica is a replica in a node that is not in the top N nodes in the preference list for a given key. The replica in this case will have in its metadata the node it should be placed on, and it will be placed there as soon as it comes back online.

Q2.8 How does the "T random tokens per node and partition by token value" partitioning strategy work? What are its limitations/issues?

Initial setup assigning random tokens, randomly placed in the ring.

When a node joins or leaves, the key ranges change, requiring expensive scans of the local persistence store and recalculation of Merkle trees.

Taking a snapshot of the key space is difficult because the ranges are random and scattered across different nodes.

Q2.9 Discuss the differences between the "T random tokens per node and equal-sized partitions" and "Q/S tokens per node, equal-sized partitions" partitioning strategies.

T random tokens per node and equal-sized partitions:

The hash space is divided into Q equally sized partitions, where Q is much larger than the number of nodes (S) and tokens (T). The tokens are used only to build a mapping function. A partition is assigned to the first N unique nodes encountered while walking the ring clockwise from the end of the partition.

This decouples data partitioning from the placement scheme, allowing for changes to placement at runtime. So nodes are linked to pre-existing partitions.

Despite the decoupling, this strategy demonstrated the worst load-balancing efficiency in comparative evaluations.

Q/S tokens per node, equal-sized partitions:

Like Strategy 2, the hash space is divided into Q equal-sized partitions. Each node is assigned a fixed number of tokens (Q/S), this ensures that every node is responsible for an equal number of partitions. When a node leaves, its tokens are redistributed to remaining nodes; when a node joins, it "steals" tokens from existing nodes to maintain the Q/S property.

Since partitions are fixed and equal-sized, they can be stored in separate files. This enables faster bootstrapping and recovery (by transferring files as units) and simplifies the archival process.

This strategy achieved the best load balancing efficiency.